KU LEUVEN

Geavanceerde Computerarchitectuur
5th Session

Dynamic parallelism and asynchronous operation

Thomas Feys, thomas.feys@kueluven.be
DRAMCO – WaveCore – ESAT
Academic year 2024/25

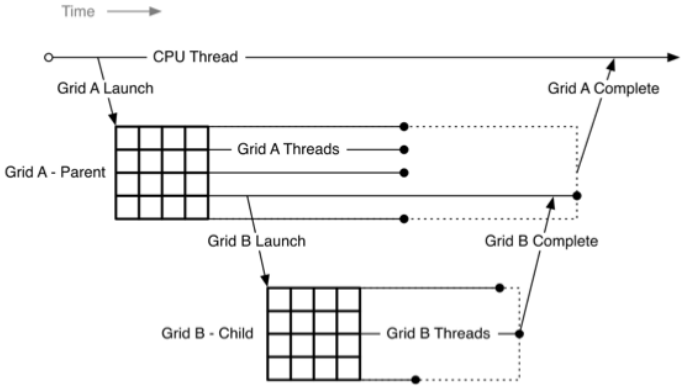
0 Outline

1. Dynamic parallelism
2. Asynchronous operation
3. Exercise

1 Outline

1. Dynamic parallelism
2. Asynchronous operation
3. Exercise

1 Spawn kernel from within kernel



1 Example

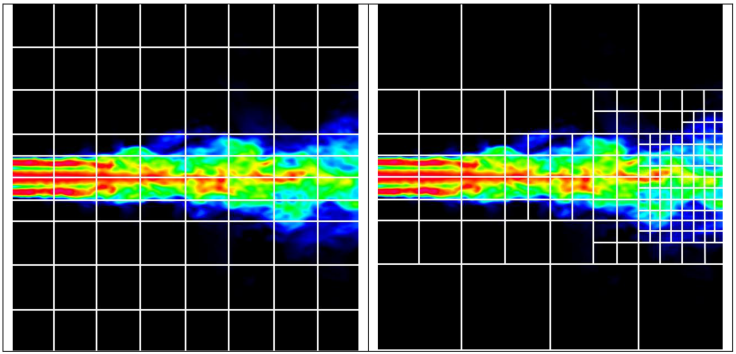
```
__global__ ChildKernel(void* data){
    //Operate on data
    }

__global__ ParentKernel(void *data){
    if (threadIdx.x == 0) {
        ChildKernel<<<1, 32>>>(data);
        cudaThreadSynchronize();
    }
    __syncthreads();
    //Operate on data
}
// In Host Code
ParentKernel<<<8, 32>>>(data);

```

1 Use case - adaptive grid generation in a computational fluid dynamics simulation

- Grid resolution is focused in regions of greatest change



2 Outline

1. Dynamic parallelism
2. Asynchronous operation
3. Exercise

2 Offload to GPU (B) without blocking the CPU (A)

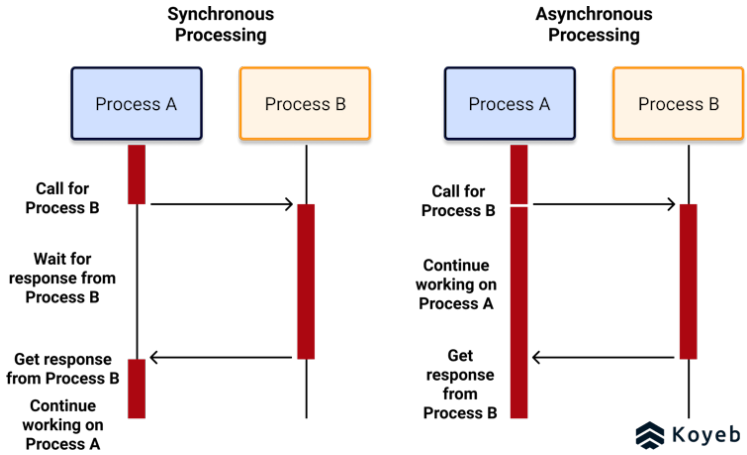


Image from Koyeb

2 (A)synchronous memory copying

```
// Copy memory synchronously (wait/block till finished)
cudaMemcpy ( dst, src, count, kind );
...
// Copy memory asynchronously (no waiting, continue with CPU code)
cudaMemcpyAsync ( dst, src, count, kind, stream );
```

A handy link to the CUDA API reference

2 Kernel invocation is asynchronous

```
1 // This is an asynchronous call by default
2 foo<<<blk, thr>>>(...);
3 ...
4 // Can synchronize manually (wait till GPU finished all of its tasks)
5 cudaDeviceSynchronize();
```

2 Kernel invocation is asynchronous

```
1  cudaEvent_t start, stop;
2  cudaEventCreate(&start);
3  cudaEventCreate(&stop);
4
5  cudaEventRecord(start);
6
7  foo<<<blk, thr>>>(...);
8
9  cudaEventRecord(stop);
10
11 // This also waits
12 cudaEventSynchronize ( stop );
```

Think of a stack (bottom up): `start`, `foo`, `stop`

2 The "GPU call stack" is ordered

Copy memory to GPU, execute, and copy back to CPU in this order and without blocking the CPU

```
1 // To GPU
2 cudaMemcpyAsync ( dst, src, count, kind, stream );
3
4 // Execute
5 foo<<<blk, thr>>>(...);
6
7 // Back to CPU
8 cudaMemcpyAsync ( dst, src, count, kind, stream );
```

2 CUDA streams

There can be more than one such "call stack", called a stream to (example):

- Distinguish between GPU calls (a call for each incoming request/user)
- Prioritize certain processing (video rendering vs. video statistics)

```
1 // Last argument is the stream
2 cudaMemcpyAsync ( dst, src, count, kind, stream );
3
4 // Last argument, again is the stream
5 foo<<<blk, thr, shared_size, stream>>>(...);
6
7 // Wait for the stream to complete
8 cudaStreamSynchronize( stream )
```

3 Outline

1. Dynamic parallelism
2. Asynchronous operation
3. Exercise

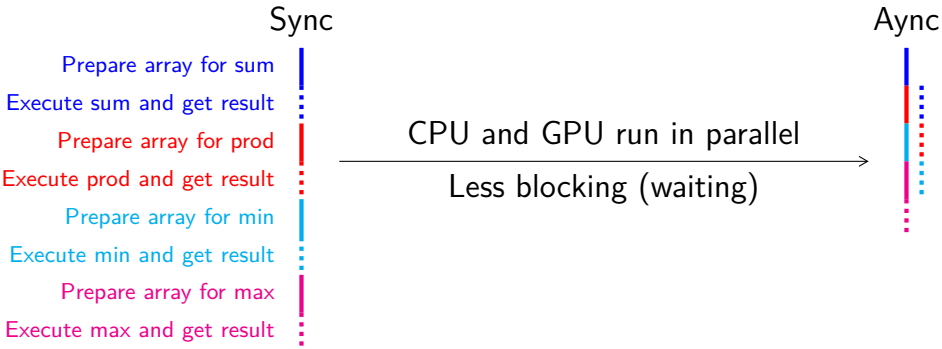
3 Sync vs async execution comparison

We will revisit reduction:

- Perform `sum`, `prod`, `min`, and `max`
- Process 4 independent arrays
- Use a for loop to generate each of the arrays
- Approach 1: do all of this in sequence, synchronously
- Approach 2: do array generation while the previous data is being asynchrounously on the GPU
- Compare the total execution time (including CPU code)

Note: for the product use values in the range $(0, 1]$ to avoid overflow errors.

3 The exercise in figure form



3 Pointers

- You already have the reduction kernel from exercise 2
- Implement summation, multiplication, and minimum detection
- Can use the same kernel and change only the computation part see below (or point to a different `__device__` function each time)

```
__global__ foo(...){
    ...
    #if KERNEL==0
    #   val = ... + ...
    #else
    #   val = ... * ...
    #endif
}
```

3 Pointers continued

- Implement a workflow where you define and populate an array on the CPU, before migrating it to the GPU and executing the kernel function.
- Do this 4 times, once for each operation: sum, prod, min, and max.
- Perhaps make 2 CPU functions - one which calls the kernel(s) synchronously, one after the other, and one that calls them while preparing the data for the next kernel.
- Call the two functions from `main()` and get the execution time using the Chrono library (see cheatsheet).
- Make sure to synchronize before stopping the timer.

3 Pointers continued

- You can experiment with NVIDIA Nsight Systems for profiling your code and seeing the different streams.